

SECTION 29 — FLAKY TEST MANAGEMENT

Purpose: Deep technical guide on non-deterministic test analysis, root-cause categorization, quarantine automation engines, flake rate metrics, and 30-day remediation strategy for Enterprise Senior SDET / Staff QA Architect leadership.

29.1 What Is a Flaky Test & Flake Metrics Math

Definition & Math

A test is **flaky** if it produces different outcomes (passes and fails) when executed on the exact same commit SHA without any code changes to either the application under test or the test script.

Flake Rate Formula

$$\text{Flake Rate (\%)} = \left(\frac{\text{Number of Tests with Flaky Outcome}}{\text{Total Tests Executed in Suite}} \right) \times 100$$

- Flaky Outcome:** A test case that failed on initial attempt but passed on automatic CI retry (Passed on Retry).
- Target SLA for Senior SDET:**
 - Healthy Enterprise Target: **< 0.5%**
 - Unhealthy Threshold (Immediate Quarantine Action): **> 2.0%**

FLAKE METRIC THRESHOLD CHART	
Metric	Operational Status
Flake Rate < 0.5%	Enterprise Green (CI Gate Trusted)
Flake Rate 0.5% - 2.0%	Amber (Requires Active Triage)
Flake Rate > 2.0%	Red (CI Trust Eroded; Immediate Action)

29.2 Root Causes Taxonomy

Category	Root Cause Mechanism	Engineered Fix
1. Async Synchronization	DOM locator evaluated before AJAX finish	Web-first auto-wait / waitF
2. Shared Data Collision	Parallel workers mutating same DB row	Dynamic UUIDs & isolated t
3. Order Dependency	Test B expects state created by Test A	Enforce test self-containme
4. Timezone / Hardcoded	Test relies on fixed date/time strings	Inject Clock / ISO UTC form

- | | | |
|--------------------------|---|-----------------------------|
| 5. Unhandled Animation | Click intercepted by fading modal/overlay | Wait for animation state st |
| 6. Unstable External API | 3rd party gateway rate limits / downtime | Stub external APIs via Wire |

29.3 Automated Flaky Test Quarantine Engine (Playwright Custom Reporter)

Production-Ready TypeScript Quarantine Reporter

```
import { Reporter, TestCase, TestResult, FullResult } from '@playwright/test/reporter';
import * as fs from 'fs';
import * as path from 'path';

export default class FlakyQuarantineReporter implements Reporter {
  private flakyTests: Array<{ title: string; location: string; retries: number }> = [];

  onTestEnd(test: TestCase, result: TestResult) {
    // If test passed after retries, it is FLAKY
    if (result.status === 'passed' && result.retry > 0) {
      console.warn( [FLAKE DETECTED] Test: "${test.title}" passed on retry #${result.retry}`);
      this.flakyTests.push({
        title: test.title,
        location: `${test.location.file}:${test.location.line}`,
        retries: result.retry,
      });
    }
  }

  onEnd(result: FullResult) {
    const reportPath = path.join(process.cwd(), 'flaky-quarantine-report.json');
    fs.writeFileSync(reportPath, JSON.stringify(this.flakyTests, null, 2));

    if (this.flakyTests.length > 0) {
      console.log(`\n Total Flaky Tests Quarantined: ${this.flakyTests.length}`);
      console.log(`Report written to ${reportPath}`);
    } else {
      console.log(`\n Zero Flaky Tests Detected in Suite!`);
    }
  }
}
```

29.4 Production-Ready TestNG IRetryAnalyzer (Java Implementation)

```
package com.deloitte.framework.retry;

import org.testng.IRetryAnalyzer;
import org.testng.ITestResult;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class RetryAnalyzer implements IRetryAnalyzer {
  private static final Logger logger = LoggerFactory.getLogger(RetryAnalyzer.class);
```

```

private int retryCount = 0;
private static final int MAX_RETRY_COUNT = 1; // Strict SLA: Max 1 retry on CI

@Override
public boolean retry(ITestResult result) {
    if (!result.isSuccess()) {
        if (retryCount < MAX_RETRY_COUNT) {
            retryCount++;
            logger.warn("⚠ Retrying failed test: [{}] | Attempt {} of {}",
                result.getName(), retryCount, MAX_RETRY_COUNT);
            return true;
        } else {
            logger.error("❌ Test [{}] FAILED after {} retries. Marking as hard failure.",
                result.getName(), MAX_RETRY_COUNT);
        }
    }
    return false;
}
}

```

29.5 30-Day Flake Elimination Master Plan

Phase 1: Quarantine & Containment (Days 1–5)

- └─ Move flaky tests to @Quarantine tag immediately.
- └─ Remove flaky tests from blocking PR gates to restore CI pipeline trust.
- └─ Configure automatic failure artifact capture (trace.zip + video).

Phase 2: Categorization & Diagnostic Audit (Days 6–15)

- └─ Audit failure logs: categorize into Async, Data Collision, Order, or Network.
- └─ Run dry-run loop: execute candidate tests 50x in parallel (`--repeat-each=50`).
- └─ Identify top 10 flaky offender test files.

Phase 3: Architectural Remediation (Days 16–25)

- └─ Replace all hardcoded sleeps (`Thread.sleep`) with explicit auto-waits.
- └─ Implement API Data Factory to isolate test data per worker thread.
- └─ Stub 3rd party external dependencies using WireMock / MSW.

Phase 4: Governance & SLA Enforcement (Days 26–30)

- └─ Set strict CI gate rule: Max 1 retry per test.
- └─ Integrate Flake Metric dashboard into Grafana/Datadog.
- └─ Establish team SLA: Untriaged flaky tests auto-generate Jira bugs assigned to feature dev team.

29.6 Anti-Patterns & Common Pitfalls

- **Setting High Retries (3-5 Retries):** Retrying tests 5 times hides real race conditions and inflates CI execution time by 300%. Max retries on CI should be **1**.
- **Using `Thread.sleep()` as a Fix:** Hardcoded sleeps slow down tests without guaranteeing element readiness on slow CI runners.

- **Ignoring Flaky Passes:** Marking a pipeline green when 5 tests passed on retry without tracking or fixing the underlying root cause.
 - **Shared Static Database Data:** Reusing fixed user records (`user_id = 100`) across concurrent workers leads to dynamic data collision failures.
-

29.7 Theory & Low-Level Execution Mechanics

Flakiness arises when asynchronous execution loops evaluate DOM states before background promises resolve.